Typestate-oriented Programming with Ad-hoc Polymorphism on the Typestate

Oliver Tušla

Introduction

We present a design of a typestate-oriented programming language.

Class instances describe their state using `clause` declarations, each intended to capture a single property of the class domain. At any point in the program, each clause in scope has a valuation: `Holds`, `Not`, or `Unknown`. For a clause `Full`, we represent its current valuation using the boxes Full, Full, and Full respectively. Traditional typestate approaches enumerate all possible combinations of properties, resulting in 2^n or 3^n number of states - an explosion in the number of declarations. **Our approach alleviates this problem by allowing users to reason about properties individually.**

Methods manipulate clause valuations. Declarations include a `makes` list specifying postconditions that must hold after method execution. Method is a set of implementations that differ only in their `takes` lists, specifying their preconditions. Methods are ad-hoc polymorphic on the state of their receiver and the states of their parameters. **This aids code organization, encapsulation, observability and performance.** We use a box like `#0` to disambiguate a method's first implementation. A method may freely alter the valuation of clauses in its own class. However, to affect the state of an instance of another class, it must do so by calling that class's methods.

Example of a Strict Domain Definition

```
class Jar {
  clause Empty; clause Full; var coins: i64

  static method new() -> Jar makes Empty, not Full {{
    return Jar { coins: 0 }
  }}

  method dump_all() makes Empty, not Full {
    takes not Empty { coins = 0 } }

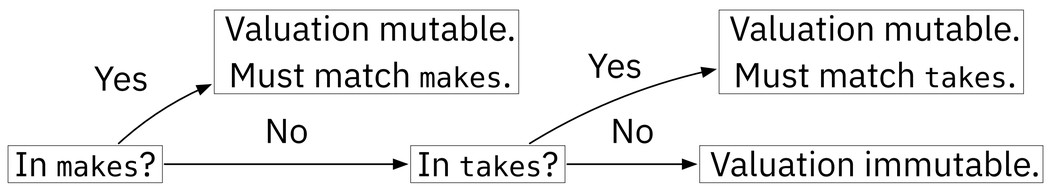
  // returns (the number of) coins that were dumped
  method dump(n: i64) -> i64 makes any Empty, any Full {
    takes not Empty, not n.Negative {
      let old: i64 = coins
      if coins - n < 0 { coins = 0 }
      else { coins = coins - n }
      return old - coins
    }}

  method fill() makes not Empty, Full {
    takes not Full { coins = MAX } }

  method add_max() makes not Empty, Full {
    takes Empty { coins = MAX } }
}
```

The `Jar` class models a strict domain, e.g., only a non-empty jar may be dumped, only a non-full jar may be filled, etc.

Clause valuation rules - (pre/post)conditions



Method implementation bodies must adhere to the clause valuation rules depicted in the diagram. Checked blocks may also refine clause valuations, using the same rules. The loop refines a jar instance. The `takes` list acts as a loop invariant, enforced at every `continue`, while the `makes` list acts as a post-condition, checked at every `break`. In the left-hand example, the `add_max()` call does not type check, and we exclude it from further discussion.

```
let j: Jar = Jar::new()                    Empty Full
loop takes not j.Full                      Empty Full
     makes not j.Empty,any j.Full{
  ✗ j.add_max()                            Empty Full
  j.fill()                                 Empty Full
  if rand_bool() {
    break                                  Empty Full
  }
  j.dump_all()                             Empty Full
}                                          Empty Full
j.dump_all()                               Empty Full
```

```
let j: Jar = Jar::new()                    Empty Full
loop takes not j.Full                      Empty Full
     makes not j.Empty {
  let n: i64 =                             Negative
    rand_i64(1, MAX)                       Positive
  j.add_positive(n)                        Empty Full
  j.dump(n)                                Empty Full
  ✗ continue                               Empty Full
}                                          Empty Full
j.dump_all()                               Empty Full
```

Ad-hoc Polymorphism on State

```
class Screen {
  clause Square; clause Wide
  pub var height: i64; pub var width: i64

  static method new_wide() -> Screen makes not Square, Wide
    {{ return Screen { height: 1080, width: 1920 } }}

  method cut_max_square() makes Square, not Wide {
    { #0
      if height < width { width = height }
      else { height = width }
    }
    takes Square {} #1
    takes Wide { width = height } #2
  }

  method render_into_square() { takes Square { .. } }

  method check_square() -> State {{
    if height == width { return State::Holds }
    else { return State::Not }
  }}

  method check_wide() -> State {{
    if height < width { return State::Holds }
    else { return State::Not }
  }}
}
```

```
var screen: Screen = Screen::new_wide()                Square Wide
if rand_bool() { screen.height = screen.width }        Square Wide
else { screen.width = screen.width * 2 }               Square Wide
decide screen Wide {                                   Square Wide
  screen.cut_max_square() #2                           Square Wide
} else {                                               Square Wide
  decide screen Square {                               Square Wide
    println("Suspicious, better safe than sorry!")
    screen.cut_max_square() #1                         Square Wide
  } else {                                             Square Wide
    screen.cut_max_square() #0                         Square Wide
  }                                                    Square Wide
}                                                      Square Wide
screen.render_into_square()                            Square Wide
```

After the screen dimensions are externally modified, all clause valuations are invalidated. We restore these properties using `decide`, which introduces two scopes, each with a distinct valuation of a clause. Subsequently, the restored valuations are leveraged by the type checker to select an appropriate implementation of the `cut_max_square()` method.

Trust and Forget statements

```
class BankAccount {
  clause AboveMin
  var balance: i64

  static method new_funded() makes AboveMin {{
    return BankAccount { balance: 100 } }}

  method pay(amount: i64) -> bool { takes AboveMin {
    balance = balance - amount                         AboveMin
    forget AboveMin                                    AboveMin
    if balance < 100 {
      balance = balance + amount                       AboveMin
      trust AboveMin                                   AboveMin
      return false                                     AboveMin
    } else {
      trust AboveMin                                   AboveMin
      return true                                      AboveMin
    }
  }}

  method bankrupt() makes not AboveMin {{ balance = 0 }}

  method gov_distrainor(debt: i64) makes AboveMin {{
    if debt + 100 > balance {                          AboveMin
      bankrupt()                                       AboveMin
      balance = 100 // social security                 AboveMin
      trust AboveMin                                   AboveMin
    } else { balance = balance - debt }                AboveMin
  }}
}
```

Clause valuations are not invalidated when modifying instance variables within the same class. To help implementors avoid unintentionally violating domain invariants, the `forget` and `trust` statements can be used, similarly to `defer` statements in other languages, as shown in the `pay()` method. Likewise, if a clause valuation introduced by a nested call conflicts with the method's declared signature, the implementor must explicitly assert that the correct property has been reestablished, as shown in `gov_distrainor()`.